EXPERIMENT-19

Implement Naive Bayes classification for Bank Loan Prediction.

Code:

```
import pandas as pd
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix
print("=" * 75)
print("            BANK LOAN PREDICTION USING NAIVE BAYES")
print("=" * 75)

# ============================================================
# STEP 1: UPLOAD DATASET
# ============================================================
uploaded = files.upload()
filename = list(uploaded.keys())[0]
print("\nFile Uploaded :", filename)

# ============================================================
# STEP 2: READ DATASET
# ============================================================
if filename.lower().endswith((".xlsx", ".xls")):
    data = pd.read_excel(filename)
else:
    data = pd.read_csv(filename)
```

```
# Remove extra spaces from column names
data.columns = data.columns.astype(str).str.strip()
print("\n" + "=" * 75)
print("                    BANK LOAN DATASET")
print("=" * 75)
print("\nDataset:")
print(data.head(15))
print("\nTotal Records :", len(data))
print("\nColumn Names:")
for i, col in enumerate(data.columns):
    print(i + 1, ".", col)


# ============================================================
# STEP 3: FIND TARGET COLUMN
# ============================================================
target = None
possible_targets = [
    "Personal Loan",
    "Personal_Loan",
    "PersonalLoan",
    "personal loan",
    "personal_loan",
    "loan",
    "Loan",
    "Loan Status",
    "Loan_Status",
    "loan_status",
    "Accepted",
    "Accepted Loan",
    "Loan Acceptance"
```

```
]
for col in data.columns:
    clean_col = str(col).strip().lower()

    for possible in possible_targets:

        if clean_col == possible.lower():
            target = col
            break
    if target is not None:
        break
# If target contains the word "loan"
if target is None:
    for col in data.columns:
        if "loan" in str(col).lower():
            target = col
            break
# If target still not found, use last column
if target is None:
    target = data.columns[-1]
    print("\nTarget column was not automatically recognized.")
    print("Using the last column as the target.")
print("\n" + "=" * 75)
print("                    TARGET INFORMATION")
print("=" * 75)
print("\nTarget Column Selected :", target)
print("\nTarget Values:")
print(data[target].value_counts(dropna=False))

# ============================================================
```

```
# STEP 4: REMOVE ROWS WITH MISSING TARGET
# ============================================================
# Target cannot be predicted if its value itself is missing
data = data.dropna(subset=[target])

print("\nRecords after removing missing target values :", len(data))

# ============================================================
# STEP 5: SEPARATE INPUT AND TARGET
# ============================================================
X = data.drop(columns=[target])
y = data[target]

# ============================================================
# STEP 6: REMOVE UNNECESSARY ID COLUMNS
# ============================================================
columns_to_remove = []
for col in X.columns:
    col_name = str(col).lower().strip()
    if col_name in ["id", "customerid", "customer id"]:
        columns_to_remove.append(col)
    if "zip" in col_name:
        columns_to_remove.append(col)
X = X.drop(
    columns=columns_to_remove,
    errors="ignore"
)

# ============================================================
# STEP 7: CONVERT CATEGORICAL COLUMNS TO NUMBERS
```

```
# ============================================================
for col in X.columns:
    if X[col].dtype == "object":
        encoder = LabelEncoder()
        X[col] = encoder.fit_transform(
            X[col].astype(str)
        )

# ============================================================
# STEP 8: CONVERT ALL FEATURES TO NUMERIC
# ============================================================
for col in X.columns:
    X[col] = pd.to_numeric(
        X[col],
        errors="coerce"
    )

# ============================================================
# STEP 9: HANDLE MISSING VALUES
# ============================================================
print("\n" + "=" * 75)
print("                HANDLING MISSING VALUES")
print("=" * 75)
missing_before = X.isnull().sum().sum()
print("\nMissing values before processing :", missing_before)
# Replace missing values with column mean
X = X.fillna(X.mean(numeric_only=True))
# If any column still contains NaN, replace with 0
X = X.fillna(0)
missing_after = X.isnull().sum().sum()
```

```
print("Missing values after processing  :", missing_after)
print("\nMissing values successfully handled.")

# ============================================================
# STEP 10: ENCODE TARGET
# ============================================================
if y.dtype == "object":
    target_encoder = LabelEncoder()

    y = target_encoder.fit_transform(
        y.astype(str)
    )
else:
    y = y.values

# ============================================================
# STEP 11: DISPLAY INPUT FEATURES
# ============================================================
print("\n" + "=" * 75)
print("                    INPUT FEATURES")
print("=" * 75)
print("\nFeatures used for prediction:")
for feature in X.columns:
    print("-", feature)
print("\nNumber of Features :", X.shape[1])

# ============================================================
# STEP 12: SPLIT DATA
# ============================================================
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
    y,
    test_size=0.30,
    random_state=42
)
print("\n" + "=" * 75)
print("                  DATA SPLITTING")
print("=" * 75)
print("\nTraining Records :", len(X_train))
print("Testing Records  :", len(X_test))

# ============================================================
# STEP 13: CREATE NAIVE BAYES MODEL
# ============================================================
model = GaussianNB()
print("\n" + "=" * 75)
print("                TRAINING NAIVE BAYES")
print("=" * 75)
model.fit(X_train, y_train)
print("\nNaive Bayes Model Training Completed Successfully!")

# ============================================================
# STEP 14: PREDICTION
# ============================================================
prediction = model.predict(X_test)
print("\n" + "=" * 75)
print("                ACTUAL VS PREDICTED")
print("=" * 75)
for i in range(min(30, len(y_test))):
    print(
```

```
        "Record", i + 1,
        "| Actual =", y_test[i],
        "| Predicted =", prediction[i]
    )

# ============================================================
# STEP 15: ACCURACY
# ============================================================
accuracy = accuracy_score(
    y_test,
    prediction
)
print("\n" + "=" * 75)
print("                    MODEL PERFORMANCE")
print("=" * 75)
print(
    "\nAccuracy :",
    round(accuracy * 100, 2),
    "%"
)

# ============================================================
# STEP 16: CONFUSION MATRIX
# ============================================================
print("\nConfusion Matrix:")
print(
    confusion_matrix(
        y_test,
        prediction
    )
```

```
)


# ============================================================
# STEP 17: FINAL RESULT
# ============================================================
print("\n" + "=" * 75)
print("            BANK LOAN PREDICTION RESULT")
print("=" * 75)
print("\nTarget Column Used :", target)
print("Model Used         : Gaussian Naive Bayes")
print("Training Samples   :", len(X_train))
print("Testing Samples    :", len(X_test))
print("Number of Features :", X.shape[1])
print(
    "Accuracy           :",
    round(accuracy * 100, 2),
    "%"
)
print("\nFinal Result:")
print(
    "The Naive Bayes algorithm successfully "
    "classified bank customers based on "
    "their loan-related information."
)
print("=" * 75)
```

Sample Input:

```
...  ======================================================================
          BANK LOAN PREDICTION USING NAIVE BAYES
     ======================================================================
     Choose Files  Bank_Pers...odelling.xlsx
     Bank_Personal_Loan_Modelling.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 350736 bytes, last modified: 8/10/2026 - 100% done
     Saving Bank_Personal_Loan_Modelling.xlsx to Bank_Personal_Loan_Modelling.xlsx
```

Sample Output:

```
================================================================================
                      BANK LOAN DATASET
================================================================================

Dataset:
    Unnamed: 0         Unnamed: 1  \
0          NaN                NaN
1          NaN                NaN
2          NaN                NaN
3          NaN                NaN
4          NaN  Data Description:
5          NaN                NaN
6          NaN                 ID
7          NaN                Age
8          NaN         Experience
9          NaN             Income
10         NaN            ZIPCode
11         NaN             Family
12         NaN              CCAvg
13         NaN          Education
14         NaN           Mortgage

                                          Unnamed: 2
0                                                NaN
1                                                NaN
2                                                NaN
3                                                NaN
4                                                NaN
5                                                NaN
6                                        Customer ID
7                   Customer's age in completed years
8                   #years of professional experience
9                Annual income of the customer ($000)
10                             Home Address ZIP code.
11                        Family size of the customer
12     Avg. spending on credit cards per month ($000)
13  Education Level. 1: Undergrad; 2: Graduate; 3:...
14             Value of house mortgage if any. ($000)
```

```
Total Records : 20

Column Names:
1 . Unnamed: 0
2 . Unnamed: 1
3 . Unnamed: 2

Target column was not automatically recognized.
Using the last column as the target.

================================================================================
                        TARGET INFORMATION
================================================================================

Target Column Selected : Unnamed: 2

Target Values:
Unnamed: 2
NaN                                                                          6
Customer ID                                                                  1
Customer's age in completed years                                            1
#years of professional experience                                            1
Annual income of the customer ($000)                                         1
Home Address ZIP code.                                                       1
Family size of the customer                                                  1
Avg. spending on credit cards per month ($000)                               1
Education Level. 1: Undergrad; 2: Graduate; 3: Advanced/Professional         1
Value of house mortgage if any. ($000)                                       1
Did this customer accept the personal loan offered in the last campaign?     1
Does the customer have a securities account with the bank?                   1
Does the customer have a certificate of deposit (CD) account with the bank?  1
Does the customer use internet banking facilities?                           1
Does the customer use a credit card issued by UniversalBank?                 1
Name: count, dtype: int64

Records after removing missing target values : 14

================================================================================
                      HANDLING MISSING VALUES
================================================================================

Missing values before processing : 14
Missing values after processing  : 0

Missing values successfully handled.

================================================================================
                        INPUT FEATURES
================================================================================

Features used for prediction:
- Unnamed: 0
- Unnamed: 1

Number of Features : 2

================================================================================
                        DATA SPLITTING
================================================================================

Training Records : 9
Testing Records  : 5

================================================================================
                    TRAINING NAIVE BAYES
================================================================================

Naive Bayes Model Training Completed Successfully!
```

```
                    ACTUAL VS PREDICTED
================================================================================
Record 1 | Actual = 5 | Predicted = 7
Record 2 | Actual = 6 | Predicted = 2
Record 3 | Actual = 3 | Predicted = 1
Record 4 | Actual = 9 | Predicted = 13
Record 5 | Actual = 11 | Predicted = 0

================================================================================
                      MODEL PERFORMANCE
================================================================================

Accuracy : 0.0 %

Confusion Matrix:
[[0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 1 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 1 0 0 0]
 [0 0 1 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 1]
 [1 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0]]

================================================================================
                BANK LOAN PREDICTION RESULT
================================================================================

Target Column Used : Unnamed: 2
Model Used         : Gaussian Naive Bayes
Training Samples   : 9
Testing Samples    : 5
Number of Features : 2
Accuracy           : 0.0 %

Final Result:
The Naive Bayes algorithm successfully classified bank customers based on their loan-related information.
--------------------------------------------------------------------------------
```